

- von Mises, R. 1964, *Mathematical Theory of Probability and Statistics* (New York: Academic Press), pp. 394ff.[3]
- Levinson, N., Appendix B of N. Wiener, 1949, *Extrapolation, Interpolation and Smoothing of Stationary Time Series* (New York: Wiley).[4]
- Robinson, E.A., and Treitel, S. 1980, *Geophysical Signal Analysis* (Englewood Cliffs, NJ: Prentice-Hall), pp. 163ff.[5]
- Bunch, J.R. 1985, "Stability of Methods for Solving Toeplitz Systems of Equations," *SIAM Journal on Scientific and Statistical Computing*, vol. 6, pp. 349–364.[6]
- de Hoog, F. 1987, "A New Algorithm for Solving Toeplitz Systems of Equations," *Linear Algebra and Its Applications*, vol. 88/89, pp. 123–138.[7]

2.9 Cholesky Decomposition

If a square matrix $\mathbf{A}$ happens to be symmetric and positive-definite, then it has a special, more efficient, triangular decomposition. *Symmetric* means that $a_{ij} = a_{ji}$ for $i, j = 0, \dots, N-1$, while *positive-definite* means that

$$\mathbf{v} \cdot \mathbf{A} \cdot \mathbf{v} > 0 \quad \text{for all vectors } \mathbf{v} \quad (2.9.1)$$

(In Chapter 11 we will see that positive-definite has the equivalent interpretation that $\mathbf{A}$ has all positive eigenvalues.) While symmetric, positive-definite matrices are rather special, they occur quite frequently in some applications, so their special factorization, called *Cholesky decomposition*, is good to know about. When you can use it, Cholesky decomposition is about a factor of two faster than alternative methods for solving linear equations.

Instead of seeking arbitrary lower and upper triangular factors $\mathbf{L}$ and $\mathbf{U}$, Cholesky decomposition constructs a lower triangular matrix $\mathbf{L}$ whose transpose $\mathbf{L}^T$ can itself serve as the upper triangular part. In other words we replace equation (2.3.1) by

$$\mathbf{L} \cdot \mathbf{L}^T = \mathbf{A} \quad (2.9.2)$$

This factorization is sometimes referred to as "taking the square root" of the matrix $\mathbf{A}$, though, because of the transpose, it is not literally that. The components of $\mathbf{L}^T$ are of course related to those of $\mathbf{L}$ by

$$L_{ij}^T = L_{ji} \quad (2.9.3)$$

Writing out equation (2.9.2) in components, one readily obtains the analogs of equations (2.3.12) – (2.3.13),

$$L_{ii} = \left(a_{ii} - \sum_{k=0}^{i-1} L_{ik}^2 \right)^{1/2} \quad (2.9.4)$$

and

$$L_{ji} = \frac{1}{L_{ii}} \left(a_{ij} - \sum_{k=0}^{i-1} L_{ik} L_{jk} \right) \quad j = i+1, i+2, \dots, N-1 \quad (2.9.5)$$

If you apply equations (2.9.4) and (2.9.5) in the order $i = 0, 1, \dots, N-1$, you will see that the L 's that occur on the right-hand side are already determined by the time they are needed. Also, only components a_{ij} with $j \geq i$ are referenced. (Since $\mathbf{A}$ is symmetric, these have complete information.) If storage is at a premium, it is possible to have the factor $\mathbf{L}$ overwrite the subdiagonal (lower triangular but not including the diagonal) part of $\mathbf{A}$, preserving the input upper triangular values of $\mathbf{A}$; one extra vector of length N is then needed to store the diagonal part of $\mathbf{L}$. The operations count is $N^3/6$ executions of the inner loop (consisting of one multiply and one subtract), with also N square roots. As already mentioned, this is about a factor 2 better than LU decomposition of $\mathbf{A}$ (where its symmetry would be ignored).

You might wonder about pivoting. The pleasant answer is that Cholesky decomposition is extremely stable numerically, without any pivoting at all. Failure of the decomposition simply indicates that the matrix $\mathbf{A}$ (or, with roundoff error, another very nearby matrix) is not positive-definite. In fact, this is an efficient way to test *whether* a symmetric matrix is positive-definite. (In this application, you may want to replace the `throw` in the code below with some less drastic signaling method.)

By now you should be familiar with, if not bored by, our conventions for objects implementing decomposition methods, so we list the object `Cholesky` as a single big mouthful. The methods `elmult` and `elsolve` perform manipulations using the matrix $\mathbf{L}$. The first multiplies $\mathbf{L} \cdot \mathbf{y} = \mathbf{c}$ for a given $\mathbf{y}$, returning $\mathbf{c}$. The second solves this same equation, given $\mathbf{c}$ and returning $\mathbf{y}$. These manipulations are useful in contexts such as multivariate Gaussians (§7.4 and §16.5) and in the analysis of covariance matrices (§15.6).

```
struct Cholesky{
```

`cholesky.h`

Object for Cholesky decomposition of a matrix $\mathbf{A}$, and related functions.

```
    Int n;
    MatDoub el;                Stores the decomposition.
    Cholesky(MatDoub_I &a) : n(a.nrows()), el(a) {
        Constructor. Given a positive-definite symmetric matrix a[0..n-1][0..n-1], construct
        and store its Cholesky decomposition,  $\mathbf{A} = \mathbf{L} \cdot \mathbf{L}^T$ .
        Int i,j,k;
        VecDoub tmp;
        Doub sum;
        if (el.ncols() != n) throw("need square matrix");
        for (i=0;i<n;i++) {
            for (j=i;j<n;j++) {
                for (sum=el[i][j],k=i-1;k>=0;k--) sum -= el[i][k]*el[j][k];
                if (i == j) {
                    if (sum <= 0.0) A, with rounding errors, is not positive-definite.
                        throw("Cholesky failed");
                    el[i][i]=sqrt(sum);
                } else el[j][i]=sum/el[i][i];
            }
        }
        for (i=0;i<n;i++) for (j=0;j<i;j++) el[j][i] = 0.;
    }
    void solve(VecDoub_I &b, VecDoub_O &x) {
        Solve the set of n linear equations  $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ , where a is a positive-definite symmetric matrix
        whose Cholesky decomposition has been stored. b[0..n-1] is input as the right-hand side
        vector. The solution vector is returned in x[0..n-1].
        Int i,k;
        Doub sum;
        if (b.size() != n || x.size() != n) throw("bad lengths in Cholesky");
        for (i=0;i<n;i++) {
            Solve  $\mathbf{L} \cdot \mathbf{y} = \mathbf{b}$ , storing  $\mathbf{y}$  in  $\mathbf{x}$ .
            for (sum=b[i],k=i-1;k>=0;k--) sum -= el[i][k]*x[k];
            x[i]=sum/el[i][i];
        }
        for (i=n-1;i>=0;i--) {
            Solve  $\mathbf{L}^T \cdot \mathbf{x} = \mathbf{y}$ .
            for (sum=x[i],k=i+1;k<n;k++) sum -= el[k][i]*x[k];
            x[i]=sum/el[i][i];
        }
    }
    void elmult(VecDoub_I &y, VecDoub_O &b) {
        Multiply  $\mathbf{L} \cdot \mathbf{y} = \mathbf{b}$ , where  $\mathbf{L}$  is the lower triangular matrix in the stored Cholesky decom-
        position. y[0..n-1] is input. The result is returned in b[0..n-1].
        Int i,j;
        if (b.size() != n || y.size() != n) throw("bad lengths");
        for (i=0;i<n;i++) {
            b[i] = 0.;
            for (j=0;j<=i;j++) b[i] += el[i][j]*y[j];
        }
    }
}
```

```

void elsolve(VecDoub_I &b, VecDoub_O &y) {
Solve  $\mathbf{L} \cdot \mathbf{y} = \mathbf{b}$ , where  $\mathbf{L}$  is the lower triangular matrix in the stored Cholesky decomposition.  $\mathbf{b}[0..n-1]$  is input as the right-hand side vector. The solution vector is returned in  $\mathbf{y}[0..n-1]$ .
    Int i,j;
    Doub sum;
    if (b.size() != n || y.size() != n) throw("bad lengths");
    for (i=0;i<n;i++) {
        for (sum=b[i],j=0; j<i; j++) sum -= el[i][j]*y[j];
        y[i] = sum/el[i][i];
    }
}

void inverse(MatDoub_O &ainv) {
Set ainv[0..n-1][0..n-1] to the matrix inverse of  $\mathbf{A}$ , the matrix whose Cholesky decomposition has been stored.
    Int i,j,k;
    Doub sum;
    ainv.resize(n,n);
    for (i=0;i<n;i++) for (j=0;j<=i;j++){
        sum = (i==j? 1. : 0.);
        for (k=i-1;k>=j;k--) sum -= el[i][k]*ainv[j][k];
        ainv[j][i] = sum/el[i][i];
    }
    for (i=n-1;i>=0;i--) for (j=0;j<=i;j++){
        sum = (i<j? 0. : ainv[j][i]);
        for (k=i+1;k<n;k++) sum -= el[k][i]*ainv[j][k];
        ainv[i][j] = ainv[j][i] = sum/el[i][i];
    }
}

Doub logdet() {
Return the logarithm of the determinant of  $\mathbf{A}$ , the matrix whose Cholesky decomposition has been stored.
    Doub sum = 0.;
    for (Int i=0; i<n; i++) sum += log(el[i][i]);
    return 2.*sum;
}
};

```

CITED REFERENCES AND FURTHER READING:

- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), Chapter I/1.
- Gill, P.E., Murray, W., and Wright, M.H. 1991, *Numerical Linear Algebra and Optimization*, vol. 1 (Redwood City, CA: Addison-Wesley), §4.9.2.
- Dahlquist, G., and Björck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §5.3.5.
- Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §4.2.

2.10 QR Decomposition

There is another matrix factorization that is sometimes very useful, the so-called *QR decomposition*,

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{R} \quad (2.10.1)$$

Here $\mathbf{R}$ is upper triangular, while $\mathbf{Q}$ is orthogonal, that is,

$$\mathbf{Q}^T \cdot \mathbf{Q} = \mathbf{1} \quad (2.10.2)$$

where $\mathbf{Q}^T$ is the transpose matrix of $\mathbf{Q}$. Although the decomposition exists for a general rectangular matrix, we shall restrict our treatment to the case when all the matrices are square, with dimensions $N \times N$.

Like the other matrix factorizations we have met (LU , SVD, Cholesky), QR decomposition can be used to solve systems of linear equations. To solve

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \quad (2.10.3)$$

first form $\mathbf{Q}^T \cdot \mathbf{b}$ and then solve

$$\mathbf{R} \cdot \mathbf{x} = \mathbf{Q}^T \cdot \mathbf{b} \quad (2.10.4)$$

by backsubstitution. Since QR decomposition involves about twice as many operations as LU decomposition, it is not used for typical systems of linear equations. However, we will meet special cases where QR is the method of choice.

The standard algorithm for the QR decomposition involves successive Householder transformations (to be discussed later in §11.3). We write a Householder matrix in the form $\mathbf{1} - \mathbf{u} \otimes \mathbf{u}/c$, where $c = \frac{1}{2} \mathbf{u} \cdot \mathbf{u}$. An appropriate Householder matrix applied to a given matrix can zero all elements in a column of the matrix situated below a chosen element. Thus we arrange for the first Householder matrix $\mathbf{Q}_0$ to zero all elements in column 0 of $\mathbf{A}$ below the zeroth element. Similarly, $\mathbf{Q}_1$ zeroes all elements in column 1 below element 1, and so on up to $\mathbf{Q}_{n-2}$. Thus

$$\mathbf{R} = \mathbf{Q}_{n-2} \cdots \mathbf{Q}_0 \cdot \mathbf{A} \quad (2.10.5)$$

Since the Householder matrices are orthogonal,

$$\mathbf{Q} = (\mathbf{Q}_{n-2} \cdots \mathbf{Q}_0)^{-1} = \mathbf{Q}_0 \cdots \mathbf{Q}_{n-2} \quad (2.10.6)$$

In many applications $\mathbf{Q}$ is not needed explicitly, and it is sufficient to store only the factored form (2.10.6). (We do, however, store $\mathbf{Q}$, or rather its transpose, in the code below.) Pivoting is not usually necessary unless the matrix $\mathbf{A}$ is very close to singular. A general QR algorithm for rectangular matrices including pivoting is given in [1]. For square matrices and without pivoting, an implementation is as follows:

```
struct QRdcmp {
```

qrncmp.h

Object for QR decomposition of a matrix $\mathbf{A}$, and related functions.

Int n;	Stored $\mathbf{Q}^T$ and $\mathbf{R}$.
MatDoub qt, r;	Indicates whether $\mathbf{A}$ is singular.
Bool sing;	Constructor from $\mathbf{A}$.
QRdcmp(MatDoub_I &a);	Solve $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ for $\mathbf{x}$.
void solve(VecDoub_I &b, VecDoub_O &x);	Multiply $\mathbf{Q}^T \cdot \mathbf{b} = \mathbf{x}$.
void qtmult(VecDoub_I &b, VecDoub_O &x);	Solve $\mathbf{R} \cdot \mathbf{x} = \mathbf{b}$ for $\mathbf{x}$.
void rsolve(VecDoub_I &b, VecDoub_O &x);	See next subsection.
void update(VecDoub_I &u, VecDoub_I &v);	Used by update.
void rotate(const Int i, const Doub a, const Doub b);	

```
};
```

As usual, the constructor performs the actual decomposition:

```
QRdcmp::QRdcmp(MatDoub_I &a)
```

qrncmp.h

```
 : n(a.nrows()), qt(n,n), r(a), sing(false) {
```

Construct the QR decomposition of $\mathbf{a}[0..n-1][0..n-1]$. The upper triangular matrix $\mathbf{R}$ and the transpose of the orthogonal matrix $\mathbf{Q}$ are stored. `sing` is set to true if a singularity is encountered during the decomposition, but the decomposition is still completed in this case; otherwise it is set to false.

```
 Int i,j,k;
 VecDoub c(n), d(n);
 Doub scale,sigma,sum,tau;
```

```

for (k=0;k<n-1;k++) {
    scale=0.0;
    for (i=k;i<n;i++) scale=MAX(scale,abs(r[i][k]));
    if (scale == 0.0) {
        sing=true;
        c[k]=d[k]=0.0;
    } else {
        Form  $\mathbf{Q}_k$  and  $\mathbf{Q}_k \cdot \mathbf{A}$ .
        for (i=k;i<n;i++) r[i][k] /= scale;
        for (sum=0.0,i=k;i<n;i++) sum += SQR(r[i][k]);
        sigma=SIGN(sqrt(sum),r[k][k]);
        r[k][k] += sigma;
        c[k]=sigma*r[k][k];
        d[k] = -scale*sigma;
        for (j=k+1;j<n;j++) {
            for (sum=0.0,i=k;i<n;i++) sum += r[i][k]*r[i][j];
            tau=sum/c[k];
            for (i=k;i<n;i++) r[i][j] -= tau*r[i][k];
        }
    }
}
d[n-1]=r[n-1][n-1];
if (d[n-1] == 0.0) sing=true;
for (i=0;i<n;i++) {
    for (j=0;j<n;j++) qt[i][j]=0.0;
    qt[i][i]=1.0;
}
for (k=0;k<n-1;k++) {
    if (c[k] != 0.0) {
        for (j=0;j<n;j++) {
            sum=0.0;
            for (i=k;i<n;i++)
                sum += r[i][k]*qt[i][j];
            sum /= c[k];
            for (i=k;i<n;i++)
                qt[i][j] -= sum*r[i][k];
        }
    }
}
for (i=0;i<n;i++) {
    r[i][i]=d[i];
    for (j=0;j<i;j++) r[i][j]=0.0;
}
}

```

Form $\mathbf{Q}^T$ explicitly.

Form $\mathbf{R}$ explicitly.

The next set of member functions is used to solve linear systems. In many applications only the part (2.10.4) of the algorithm is needed, so we put in separate routines the multiplication $\mathbf{Q}^T \cdot \mathbf{b}$ and the backsubstitution on $\mathbf{R}$.

```

qrncmp.h void QRncmp::solve(VecDoub_I &b, VecDoub_O &x) {
    Solve the set of n linear equations  $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ .  $\mathbf{b}[0..n-1]$  is input as the right-hand side vector,
    and  $\mathbf{x}[0..n-1]$  is returned as the solution vector.
    qtmult(b,x);          Form  $\mathbf{Q}^T \cdot \mathbf{b}$ .
    rsolve(x,x);          Solve  $\mathbf{R} \cdot \mathbf{x} = \mathbf{Q}^T \cdot \mathbf{b}$ .
}

void QRncmp::qtmult(VecDoub_I &b, VecDoub_O &x) {
    Multiply  $\mathbf{Q}^T \cdot \mathbf{b}$  and put the result in  $\mathbf{x}$ . Since  $\mathbf{Q}$  is orthogonal, this is equivalent to solving
     $\mathbf{Q} \cdot \mathbf{x} = \mathbf{b}$  for  $\mathbf{x}$ .
    Int i,j;
    Doub sum;
    for (i=0;i<n;i++) {
        sum = 0.;

```